CSC 4553

Fall 2026 - Day 5

Admin

- Quiz 2 due today, 18:00

Process basics

Questions to consider

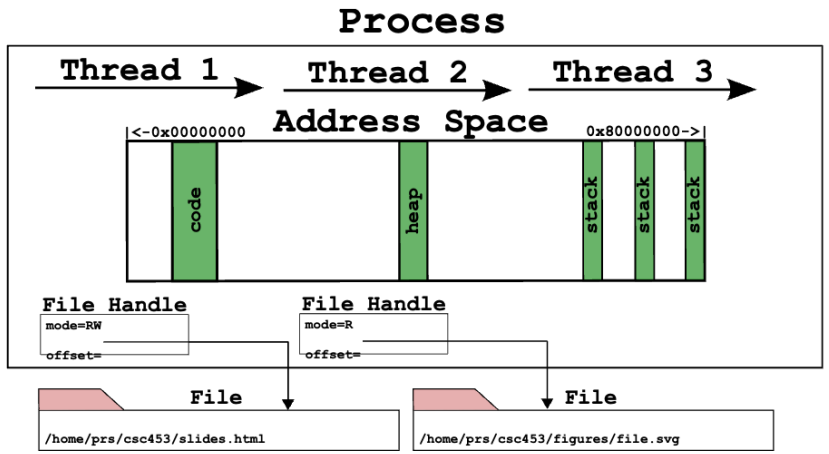
- What do processes contain?
- How does the OS run multiple processes at the same time?
- How are processes laid out in memory?
- How does the OS store information about each process?

Processes

- Most fundamental OS abstraction
  - → Processes organize information about other abstractions and represent a single thing the computer is “doing”
- When you run an executable program (passive), the OS creates a process == a running program (active)
- One program can be multiple processes

Process organization

- Unlike threads, address spaces and files, processes are not tied to a hardware component. Instead, they contain other abstractions
- Processes contain:
  - → one or more **threads**,
  - → an **address space**, and
  - → zero or more open **file handles** representing files



Multiprogramming

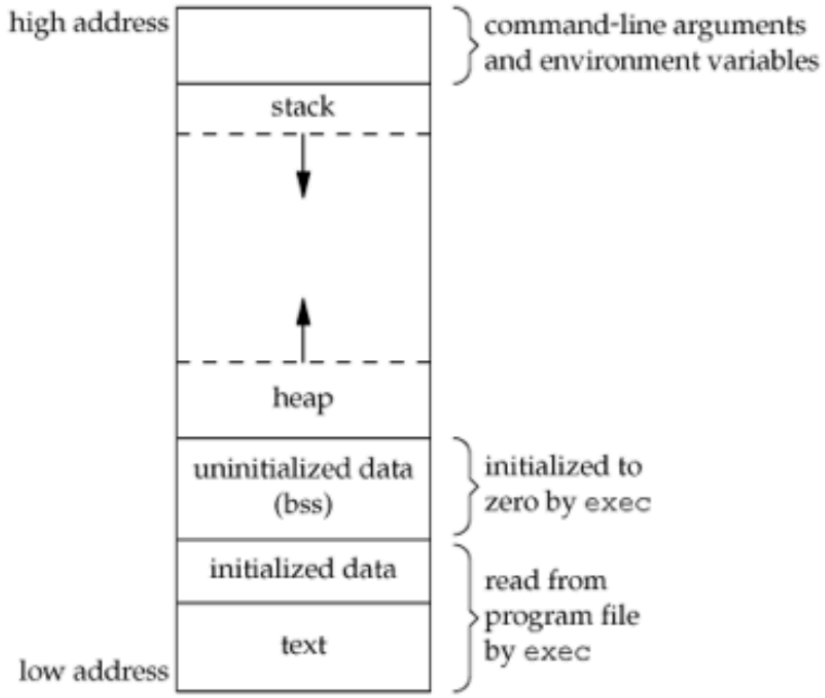
- Processes are the core abstraction that allows for **multiprogramming**: the illusion of concurrency
- OS timeshares CPU across multiple processes: virtualizes CPU
- OS has a CPU scheduler that picks one of the many active processes to execute on a CPU
- Policy:
  - → **which** process to run
- Mechanism:
  - → how to **context switch** between processes

Process's view of the world

- Own memory with consistent addressing (divorced from physical addressing)
- It has exclusivity over the CPU: It doesn't have to worry about scheduling
- Conversely, it doesn't know when it will be scheduled, so real time events require special handling
- Has some identity: `pid`, `gid`, `uid`
- Has a set of services available to it via the OS
  - → Data (via file system)
  - → Communication (sockets, IPC)
  - → More resources (e.g., memory)

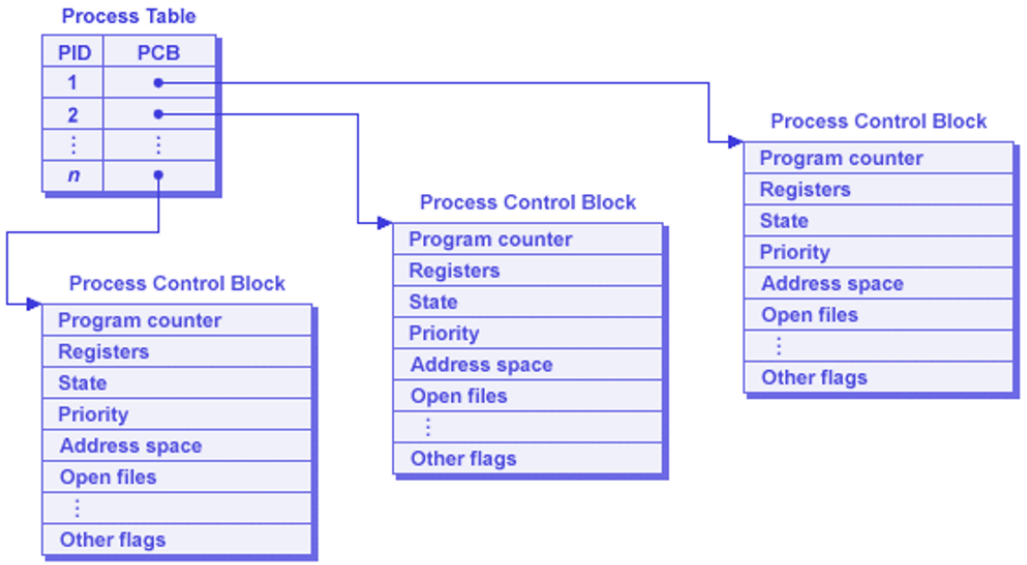
Process memory layout

- **Text** segment: machine instructions; shareable between identical processes; read-only
- **Data** segment: for initialized data; e.g., `int count = 99;`
- **BSS** (block started by symbol) segment: uninitialized data; e.g., `int sum[10];`
- **Heap**: dynamic memory allocation
- **Stack**: initial arguments and environment; stack frames



OS's view of the (process) world

- Data for each process is held in a data structure known as a **Process Control Block**
- Partitioned memory:
  - → dedicated & shared address space
  - → perhaps non-contiguous
- **Process table** holds PCBs



Where does it live?

- **Think (2 min)**: which segment holds each labeled item: **text**, **data**, **BSS**, **heap**, or **stack**?

```c
int   count = 99;              // (a) count
char  buf[4096];               // (b) buf
int main(int argc, char **argv) {
    int i = 0;                 // (c) i
    char *p = malloc(1024);    // (d) p    (e) what p points to
    printf("%d\n", count);     // (f) the literal "%d\n"
    return 0;                  // (g) the instructions of main() itself
}
```

- **Discuss (2 min)**: compare answers. Which ones did you disagree on?
- **Question (1 min)**: you run this same program twice, so there are two processes. Which segments could the OS share between them, and which have to be private?

What isn’t clear?

Comments? Thoughts?

Process state and scheduling

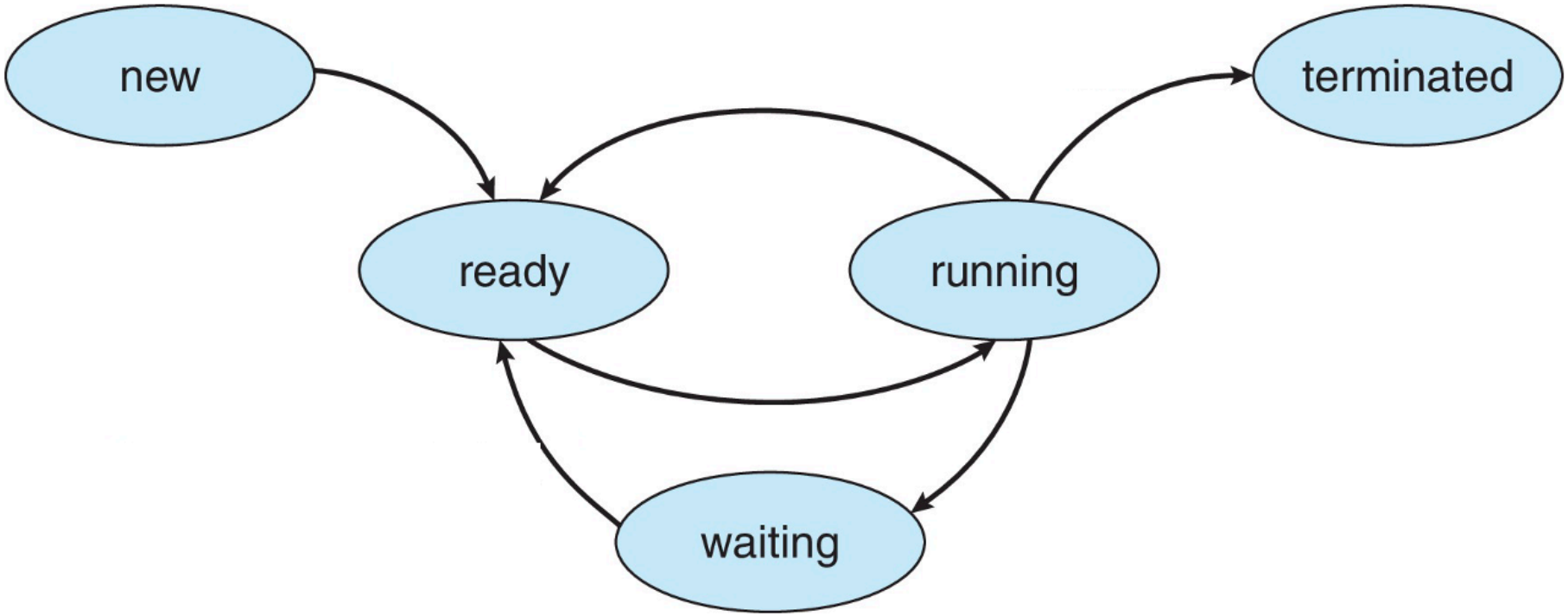
Questions to consider

- What are the different process states and what causes transitions?
- What is a context switch?
- What are the two general categories of processes and how do they differ?

Process states

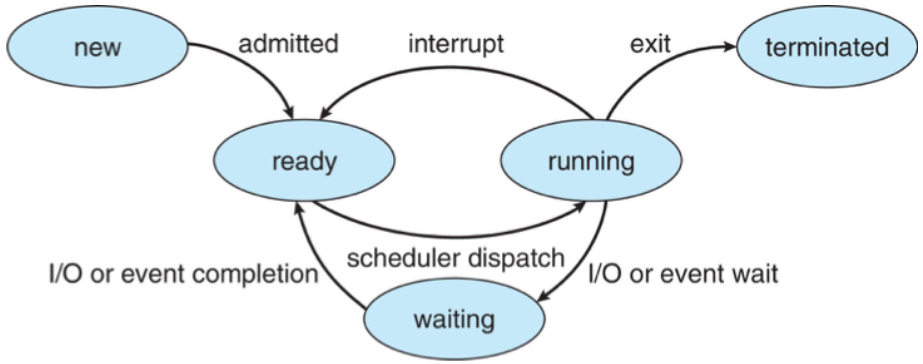
- As a process executes, it changes *state*
  - → **New**: The process is being created
  - → **Running**: Instructions are being executed
  - → **Waiting**: The process is waiting for some event (typically I/O or signal handling) to occur
  - → **Ready**: The process is waiting to be assigned to a processor
  - → **Terminated**: The process has finished execution

Process state transitions



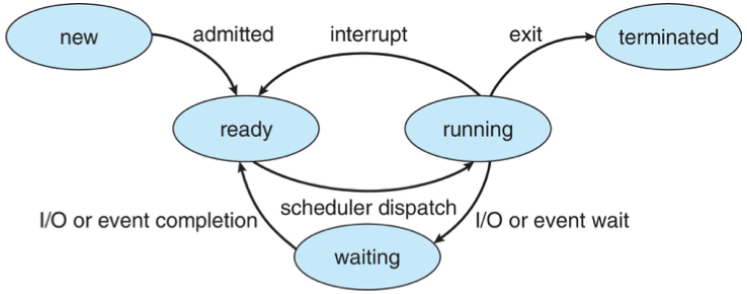
Process state transitions (cont'd)

- Running process can move from running to terminated (exit or killed), moved to ready (time slice up), or blocked (signaled to wait, I/O)
- Which state transitions could happen with these expensive actions?
  - → Compute a new RSA key?
  - → Find the largest value in a 1TB of data?



State transition example

- **Think (2 min)**: a running process calls `read()` on a file. Trace its state transitions from that moment until it gets its data back. Name what causes **each** transition
- **Discuss (2 min)**: compare traces. Did you agree on how many transitions there are, and on what triggers each one?
- **Share (1 min)**: now the data is already in memory (the kernel cached it earlier), so no disk access is needed. Does your trace change?



Process scheduling

- **OS process scheduler** selects among available processes for next execution on CPU core
- Goal?
  - → Maximize CPU use, quickly switch processes onto CPU core
- Maintains **scheduling queues** of processes
  - → **Ready queue**: set of all processes residing in main memory, ready and waiting to execute
  - → **Wait queues**: set of processes waiting for an event (i.e., I/O)
- Processes migrate among the various queues over their lifetime

Context switching

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a **context switch**
- Context of a process represented in the PCB
- Context-switch time is **pure overhead**; the system does no useful work while switching
  - → The more complex the OS and the PCB → the longer the context switch
- Time dependent on hardware support
  - → Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once

Context switching overhead

- On the order of microseconds on modern hardware, used to be milliseconds
- If not done intelligently, you can spend more time context-switching than actual processing
- Question: Why shouldn't processes control context switching?

Scheduling basics

- Scheduler usually makes the transition decisions; hides the details from the process/user
- Processes often characterized as one of two types by what state they spend most of their time in
  - → **I/O bound**: work is dependent on I/O; e.g., browser, db, media streaming
  - → **CPU bound**: work is dependent on CPU; e.g., scientific apps, cryptography
  - → Why does this matter?
    - Understanding which your process is allows for optimization
      - CPU-bound? Faster CPU, parallelize.
      - I/O? Faster I/O devices, use async
- Scheduler must balance CPU- & I/O-bound processes

What isn’t clear?

Comments? Thoughts?